Loan System Design

1. System Structure
2. Data Structure
3. API Design
   - 3.1 Create Loan
     - 3.1.1 API Information
     - 3.1.2 Functionality
     - 3.1.3 Implementation Details
   - 3.2 Query Loan
     - API Information
   - 3.3 Approve Loan
     - 3.3.2. Functionality
     - 3.3.3. Implementation Details
   - 3.4 Invest Loan
     - 3.4.1 API Information
     - 3.4.2 Functionality
     - 3.4.3 Implementation Details
   - 3.5 Disburse Loan
     - 3.5.1 API Information
     - 3.5.2 Functionality
     - 3.5.3 Implementation Details
4. Test
   - Prepare For The Test
   - 4.1 Create Loan API
   - 4.2 Get Loan API
   - 4.3 Approve Loan API
   - 4.4 Invest Loan API
   - 4.5 Disburse Loan API
   - Additional Notes

This project prioritizes core business logic, data consistency, and maintainability. To keep the implementation lightweight, some production-level components, such as Redis and scheduled retry jobs, have been omitted and can be added if needed.

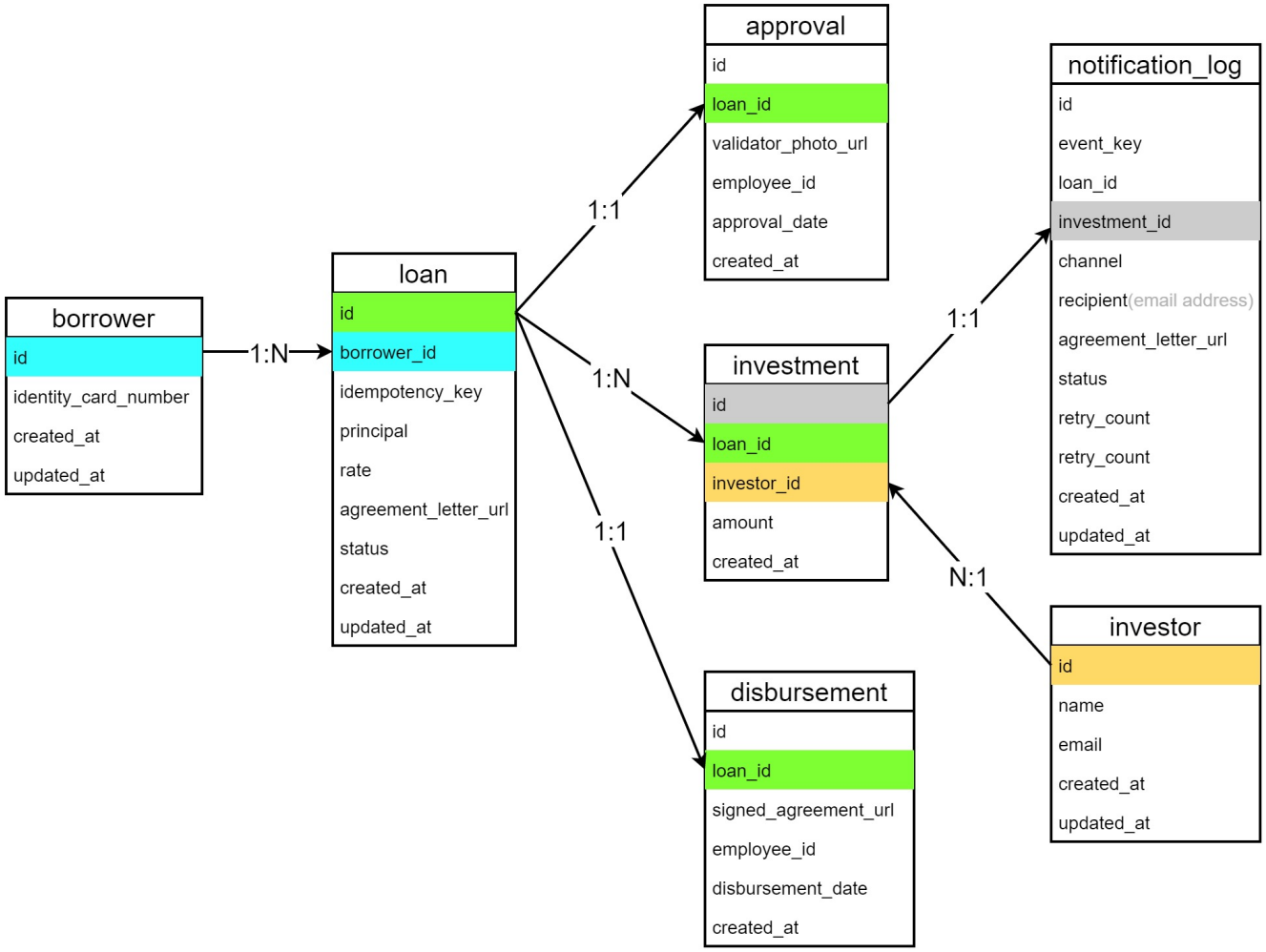
1. System Structure

LoanSystem manages the loan lifecycle, from creation and approval to investment and disbursement. The project uses Go 1.22, Gin, GORM, and MySQL.

```
HTTP → Gin Handler → Service → Repository → MySQL
```

- **Handler**（`internal/handler`）：Registers routes, binds JSON requests, parses path parameters, and maps business errors to HTTP status codes and JSON responses.
- **Service**（`internal/service`）：Enforces business rules and coordinates Repository operations. Core business logic, concurrency control, and idempotency handling are implemented at this layer.
- **Repository**（`internal/repository`）：Encapsulates GORM queries and database write operations.
- **Model / DTO / Mapper**：Define the persistence structures, HTTP request and response formats, and conversions between DTOs and Models, respectively.

2. Data Structure



3. API Design

3.1 Create Loan

3.1.1 API Information

- `POST /api/v1/loans/create`
- Headers: `Idempotency-Key`, used for idempotency validation. Example: `3f2a9c1e-7b4d-4e8a-9f21-5c6d8e0a1b2c`
- JSON Body

CreateLoanRequest Plain Text

```
{
  "borrowerId": 1,
  "principal": 100000,
  "AgreementLetterURL": "https://example.com/photos/agreement.jpg",
  "rate": 0.05,
  "roi": 0.08
}
```

3.1.2 Functionality

Create a loan and initialize its status to `PROPOSED`

3.1.3 Implementation Details

- **Parameter Validation**:Ensure all required fields are provided and conform to the expected formats. Principal must be a positive integer.
- **Business Logic Validation**:The Borrower must already exist in the database.
- **Idempotency**:To handle duplicate submissions (e.g., repeated button clicks or network retries), the client includes an `Idempotency-Key` header. The database enforces uniqueness on `idempotency_key`. Before inserting a new loan, the service checks whether the key already exists.
- **Security**:Validate user-supplied text inputs, such as URLs, to mitigate injection and XSS risks.

3.2 Query Loan

API Information

- `GET /api/v1/loans/{id}`
- Path Parameter: `loan_id` (positive integer)

3.3 Approve Loan

API Information

- `POST /api/v1/loans/{id}/approve`

- Path Parameter: `loan_id` (positive integer)
- JSON Body

ApproveLoanRequest Markdown

```
{
    "validatorPhotoUrl": "https://example.com/validator.jpg",
    "employeeId": "E-001",
    "approvalDate": "2026-10-19T00:00:00Z"
  }
```

3.3.2. Functionality

Approve a loan and update its status to `APPROVED`

3.3.3. Implementation Details

- **Parameter Validation**:Ensure all required fields are provided and conform to the expected data types.
- **Business Logic Validation**:Only loans with the `PROPOSED` status can be approved.
- **Concurrency Control**:
  - Status validation, Approval record insertion, and Loan status updates are performed within a Service-layer transaction. A row-level lock on the Loan table serializes concurrent approval requests for the same loan. Duplicate approval requests are rejected with `409 Conflict` when the status check detects that the loan has already been approved.
  - The `loan_id` field in the Approval table has a unique index, ensuring that each loan can have at most one approval record.

3.4 Invest Loan

3.4.1 API Information

- `POST /api/v1/loans/{id}/invest`
- Path Parameter: `loan_id` (positive integer)
- Body：JSON `InvestRequest`

InvestRequest Markdown

```
{
  "investorId": 1,
  "amount": 400.00
}
```

3.4.2 Functionality

Investors can invest in a loan through this endpoint. When the total investment reaches the loan principal, the loan status changes to `INVESTED`, and an email notification is sent to the investor.

3.4.3 Implementation Details

- **Parameter Validation**: Ensure all required fields are provided and conform to the expected data types.
- **Business Logic Validation**:
  - The Investor must already exist in the database.
  - Only loans with the `APPROVED` status can receive investments.
  - If the total investment is less than the loan principal, the loan status remains `APPROVED`.
  - If the total investment equals the loan principal, the loan status changes to `INVESTED`, and email notifications are sent to all investors.
  - If the total investment exceeds the loan principal, the investment is rejected.
- **Concurrency Control**:
  - Investment record creation and Loan status updates are performed within a Service-layer transaction. All changes are rolled back if the transaction fails.
  - Before processing an investment, the service locks the corresponding Loan row using SELECT ... FOR UPDATE. It then calculates the total investment and inserts the new record within the transaction. In databases that support row-level locking, concurrent requests for the same Loan must wait for the preceding transaction to complete before recalculating the total investment, preventing multiple requests from being approved based on the same stale balance.
- **Asynchronous Email Notifications**:
  - Email notification logic is decoupled from the investment process. Once the Loan status

changes to `INVESTED`, the email task is executed asynchronously, and the email delivery status is recorded in the database. Email delivery failures do not affect the investment operation.

- ~~Potential Enhancement: To improve delivery reliability, a scheduled task could periodically scan for failed email deliveries and retry sending them. This feature is not currently implemented.~~
- Worker: A background scheduler task will scan to find the unsent email (status = PENDING).
- Log Status Transitions:
  - PENDING → SEDNING → (send succeeded) → SENT
  - PENDING → SEDNING → (send failed & retry count <=3) → PENDING → (retry succeeded) → SENT
  - PENDING → SEDING → (send failed & retry count >3) → FAILED
- The current implementation uses a mock interface and does not send actual emails.

3.5 Disburse Loan

3.5.1 API Information

- `POST /api/v1/loans/{id}/disburse`
- Path Parameter: `loan_id` (positive integer)
- JSON Body

DisburseRequest Markdown

```
{
    "signedAgreementUrl": "https://example.com/signed-agreement.pdf",
    "employeeId": "E-001",
    "disbursementDate": "2026-10-19T00:00:00Z"
}
```

3.5.2 Functionality

Employees can disburse a loan through this endpoint, updating its status to `DISBURSED`

3.5.3 Implementation Details

- **Parameter Validation**: Ensure all required fields are provided and conform to the expected

data types.

- **Business Logic Validation**:
  - Only loans with the `INVESTED` status can be disbursed.
  - signedAgreementUrl must be a valid HTTP(S) URL with a host, and its path must have a .pdf, .jpg, or .jpeg extension (case-insensitive).
- **Concurrency Control**:
  - Start a database transaction and lock the Loan row using FindByIDForUpdate.
  - Create the Disbursement record, then update the Loan status to `DISBURSED` .
  - Roll back the transaction if any step fails.
- **Security**: Validate user-supplied text inputs, such as URLs, to mitigate injection and XSS risks.

4. Test

Prepare For The Test

Before testing，it is necessary to insert some data into database.

Borrower Table

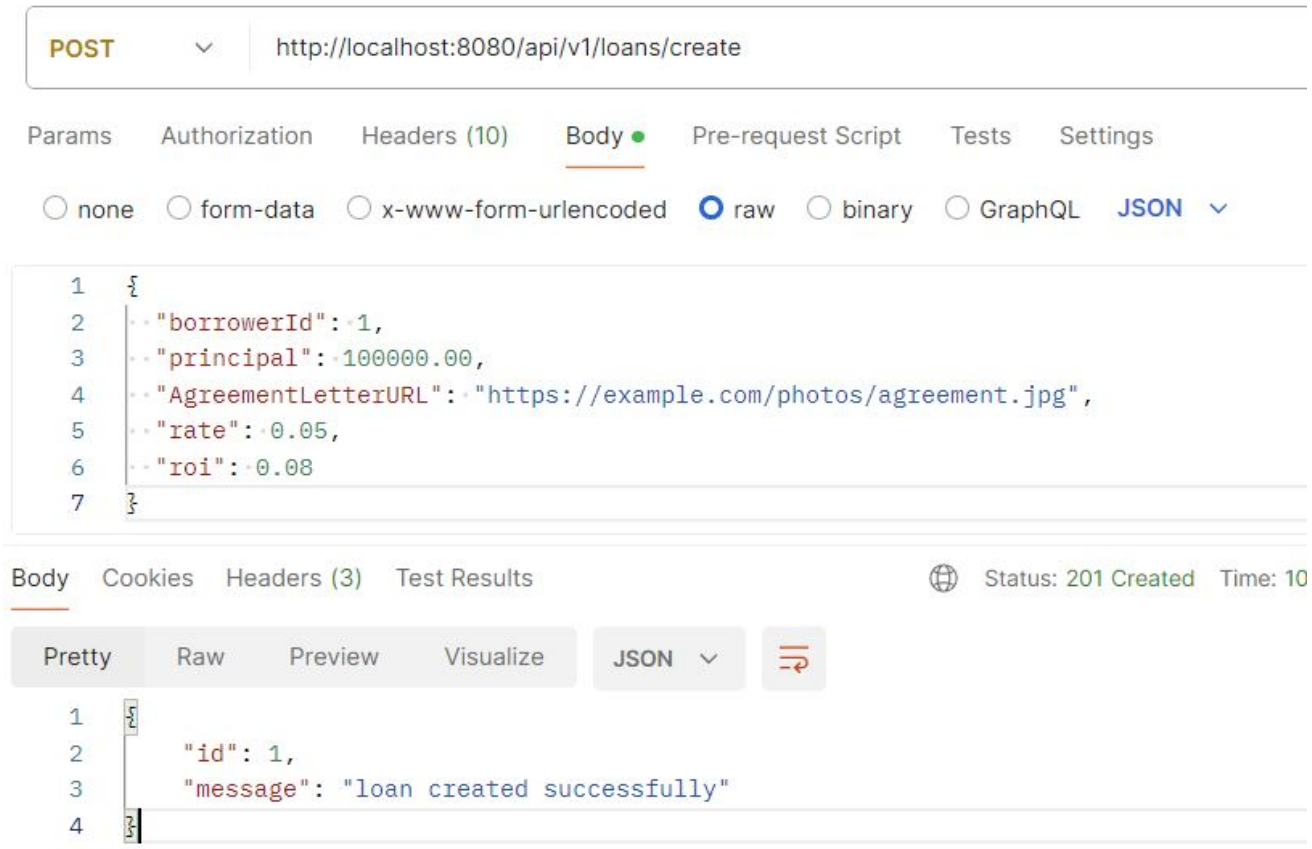
ID	IdentityCardNumber
1	`3171010101800001`
2	`3273024112850002`

Investor Table

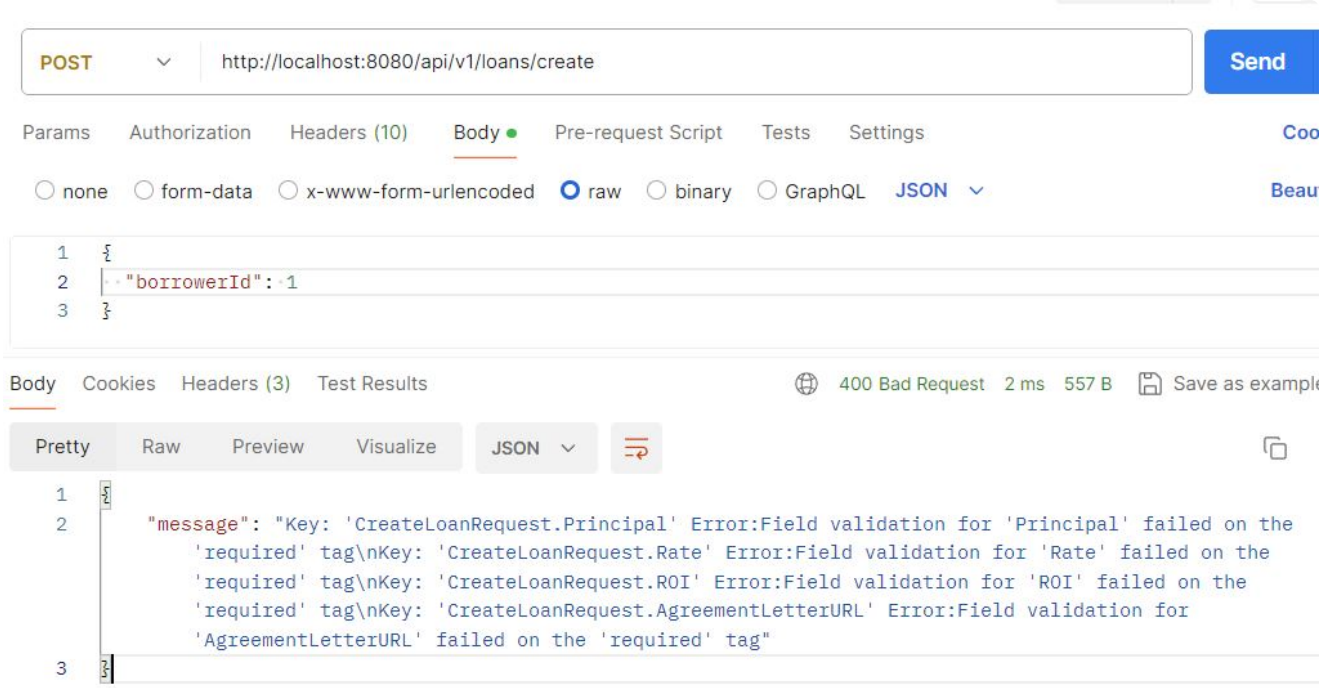
ID	Name	Email
1	Test Investor One	investor1@example.com
2	Test Investor Two	investor2@example.com

4.1 Create Loan API

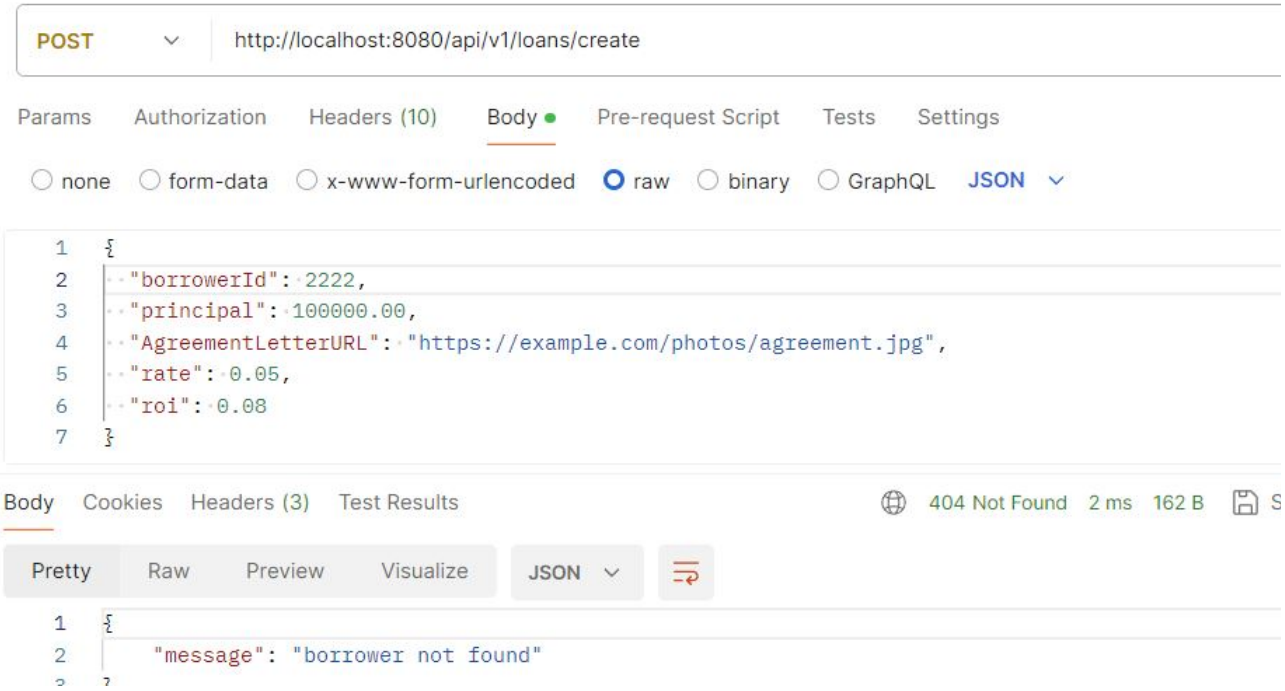
Test Case 1: Successful Loan Creation



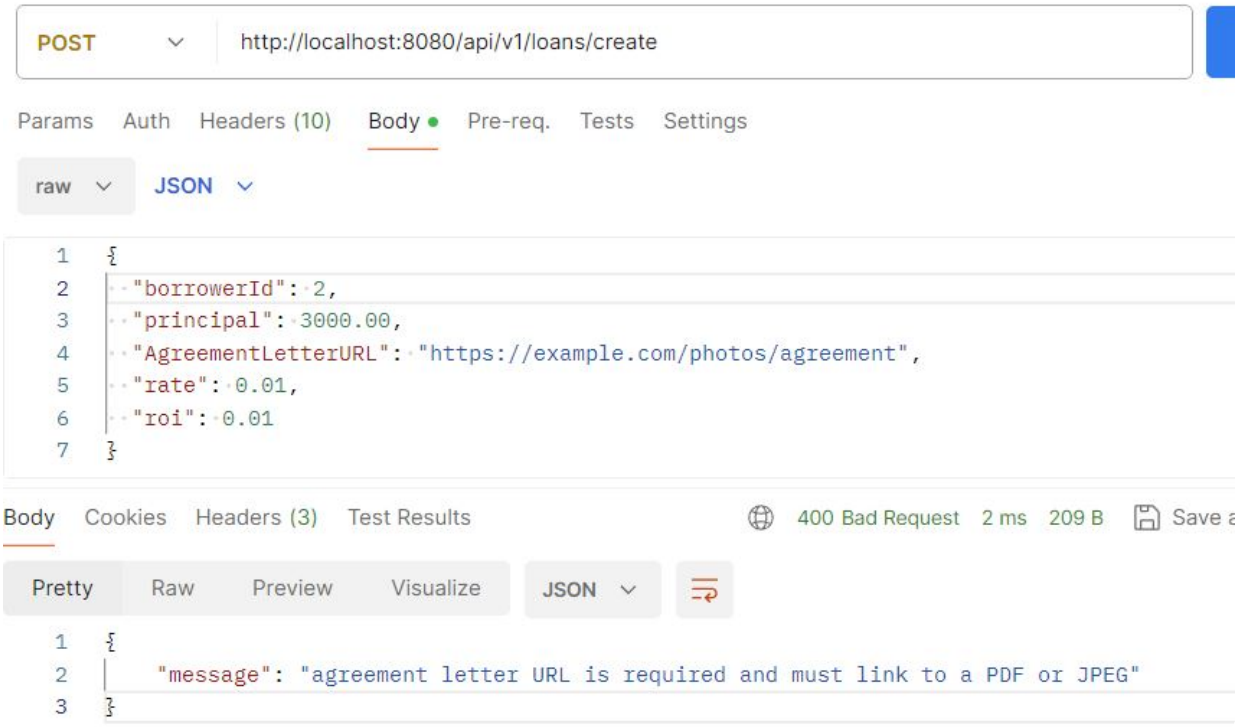
Test Case 2: Missing Required Parameters



Test Case 3: Borrower Does Not Exist

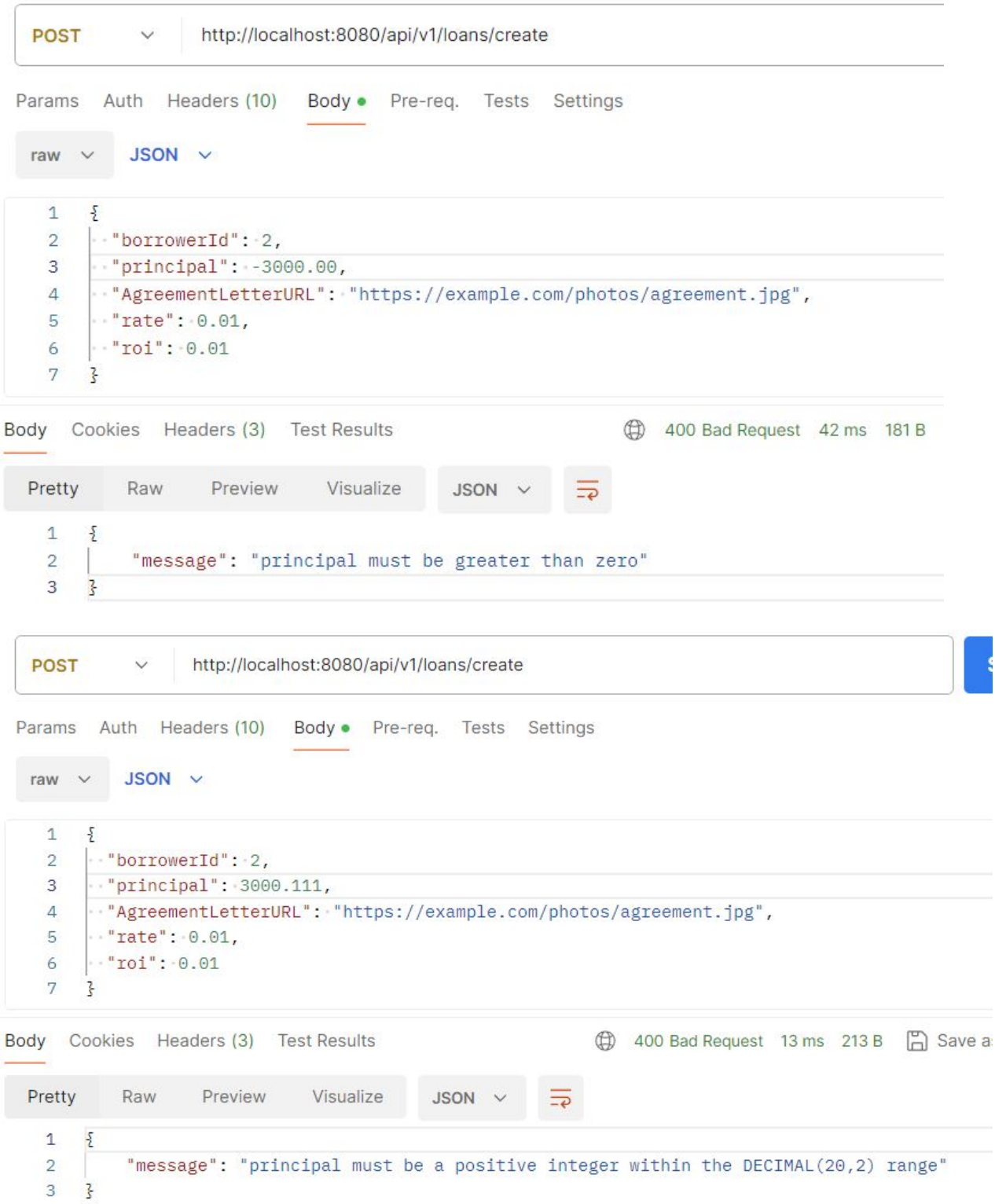


Test Case 4: Invalid URL

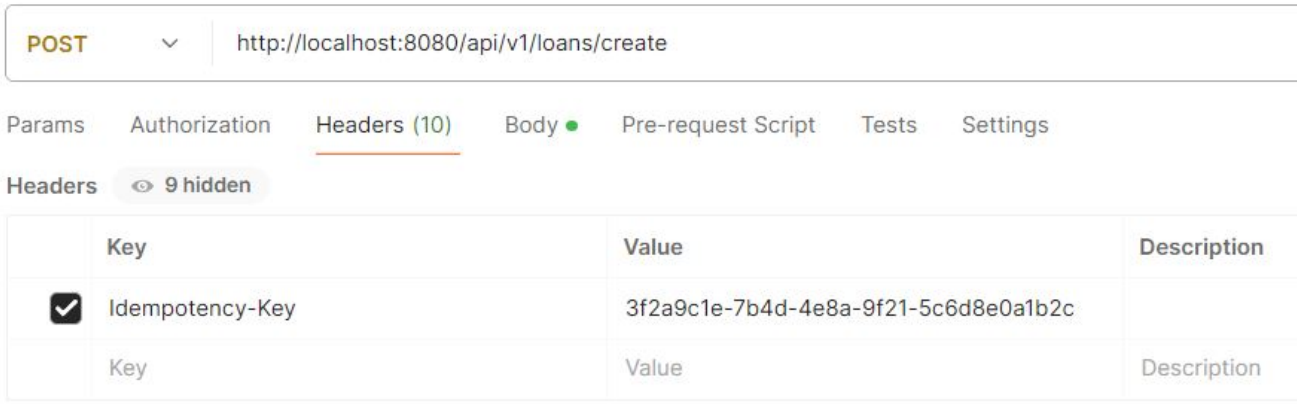


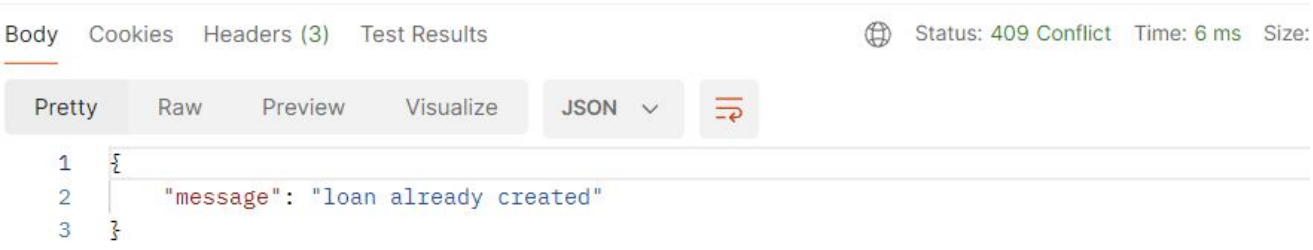
Test Case 5: Invalid Principal

The Principal must be greater than zero



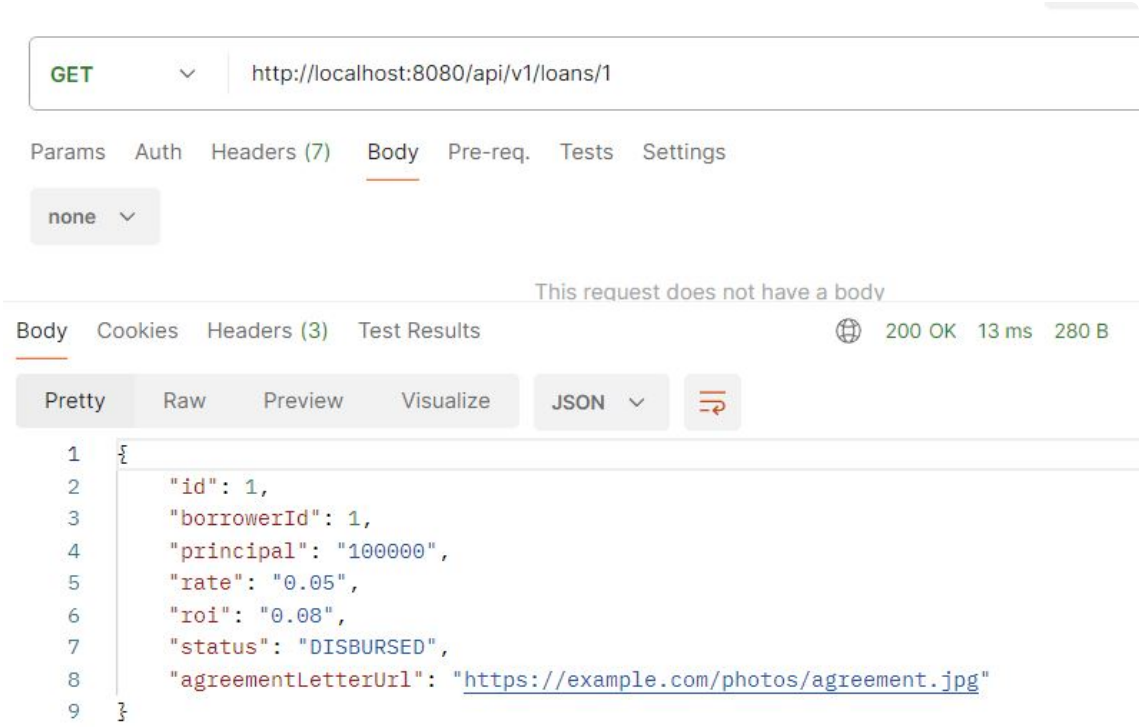
Test Case 6: Duplicate Idempotency Key



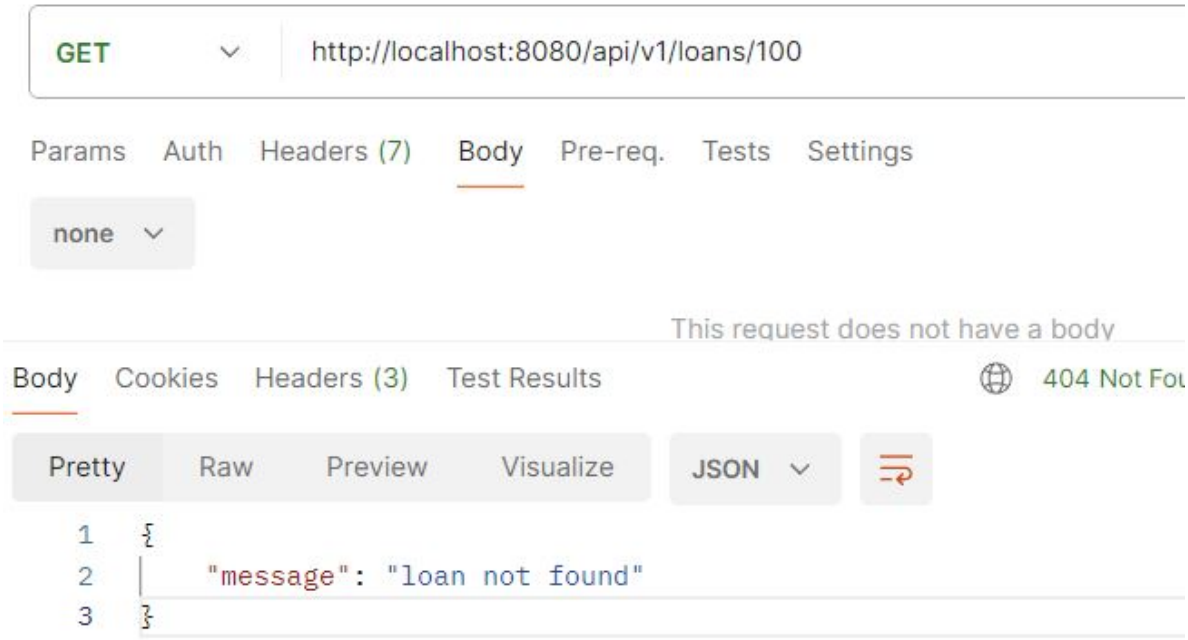


4.2 Get Loan API

Test Case 1: Successful Get Loan



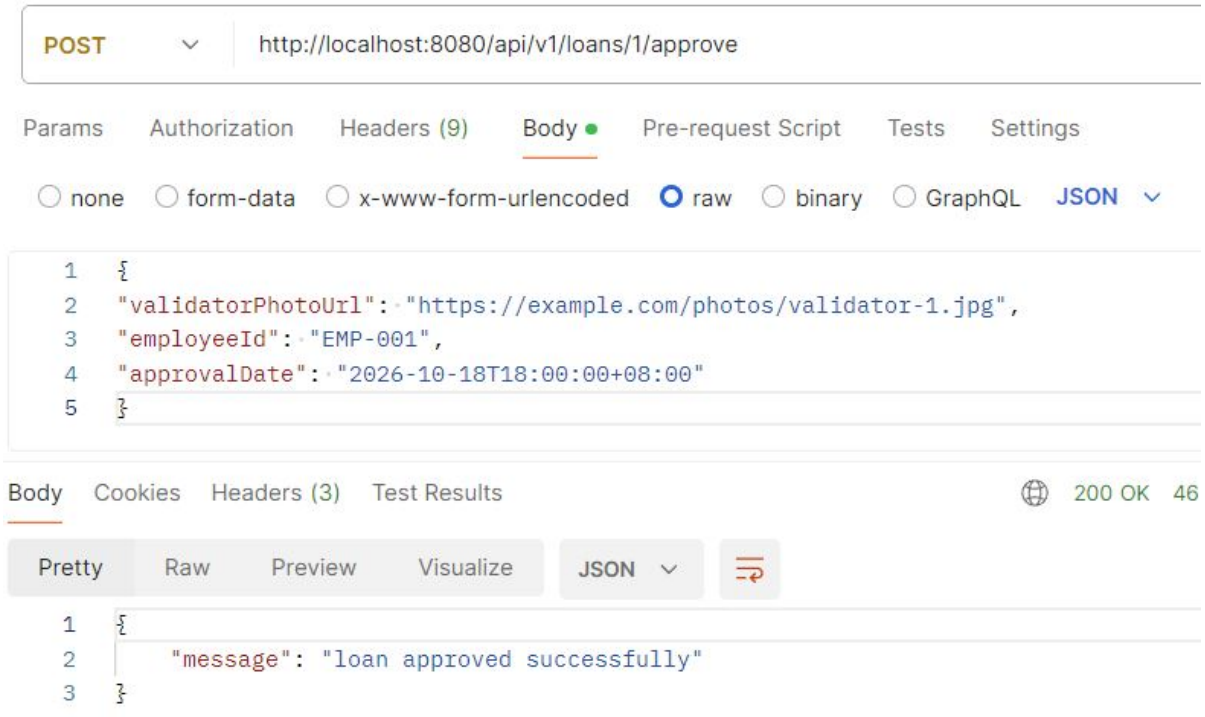
Test Case 2: Loan Not Exists



4.3 Approve Loan API

Test Case 1: Successful Loan Approval

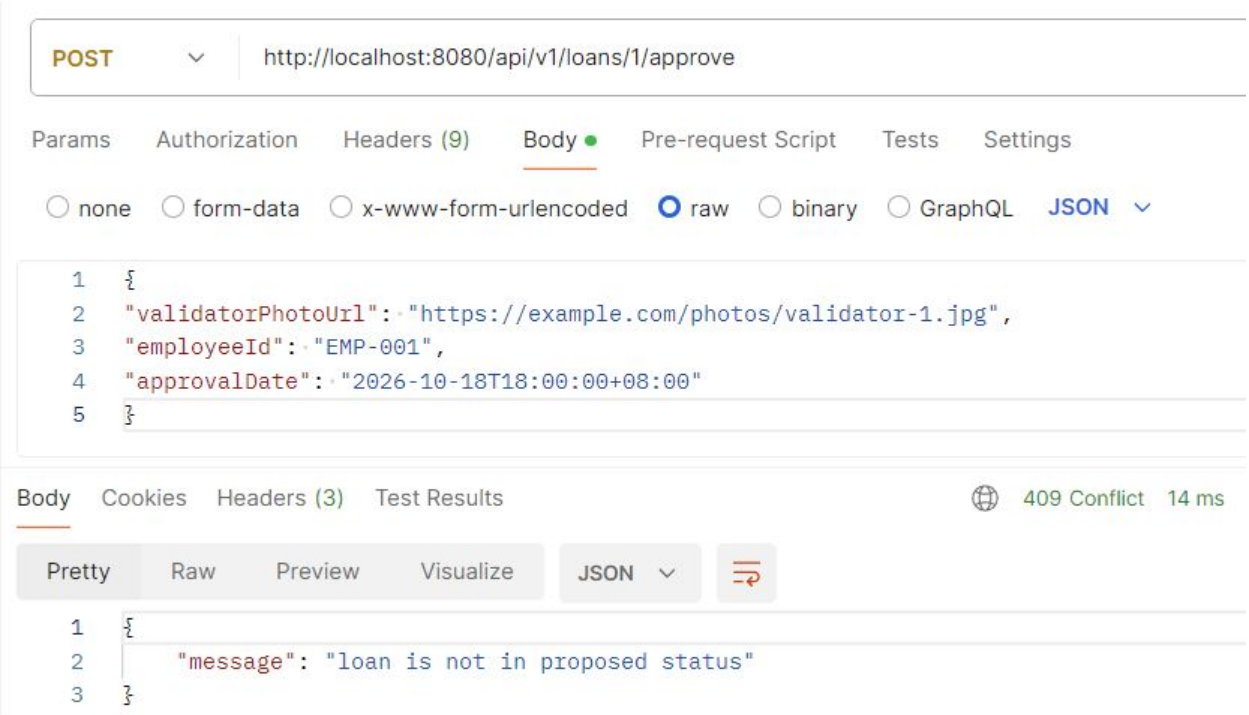
From `PROPOSED` to `APPROVED`



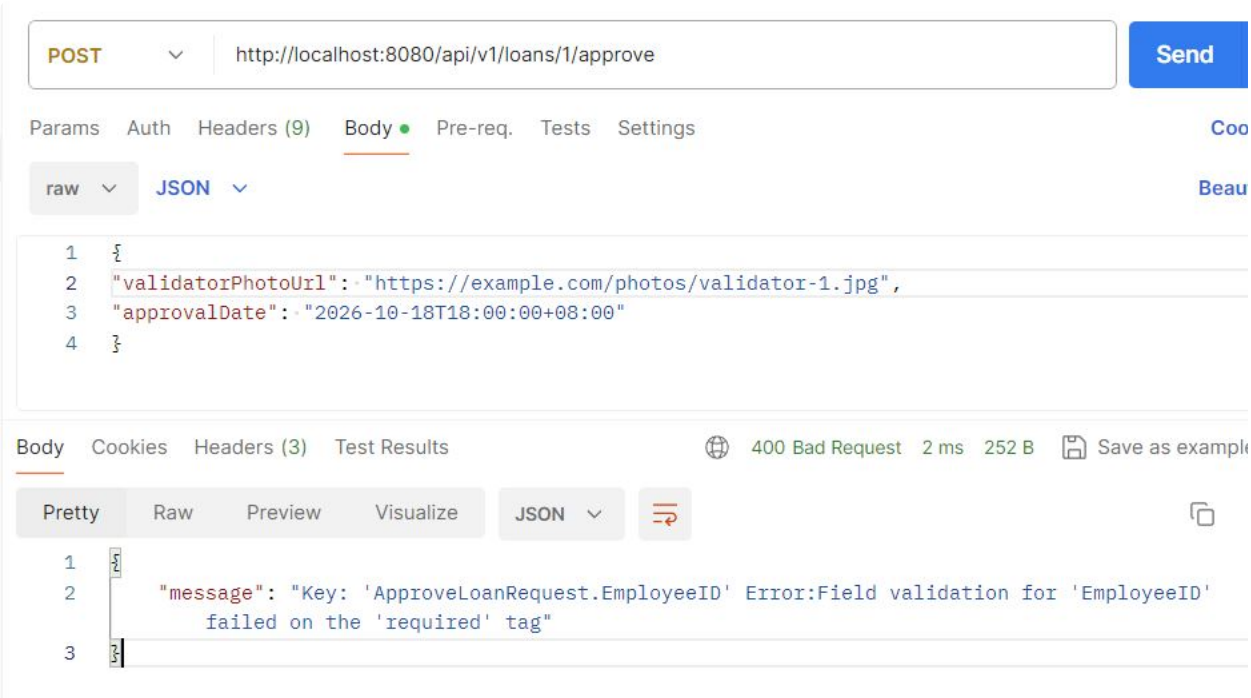
Test Case 2: Invalid Loan Status

This loan status is already `APPROVED`

Can not transfer from `APPROVED` to `APPROVED`



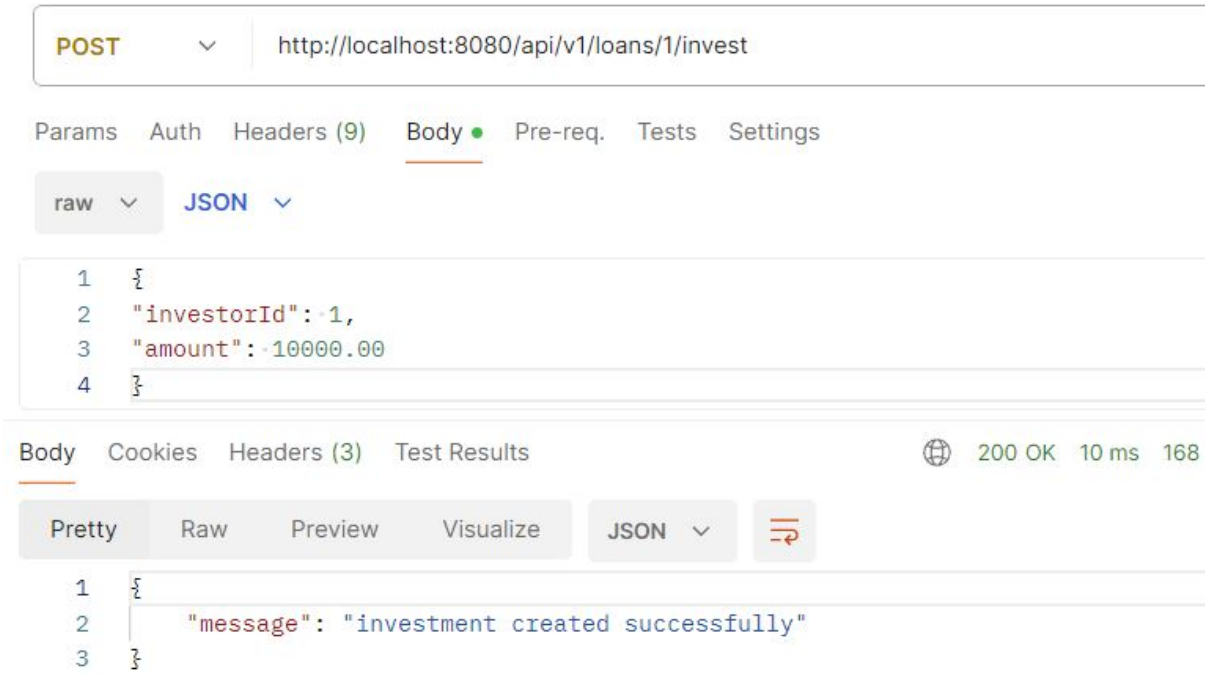
Test Case 3: Missing Required Parameters



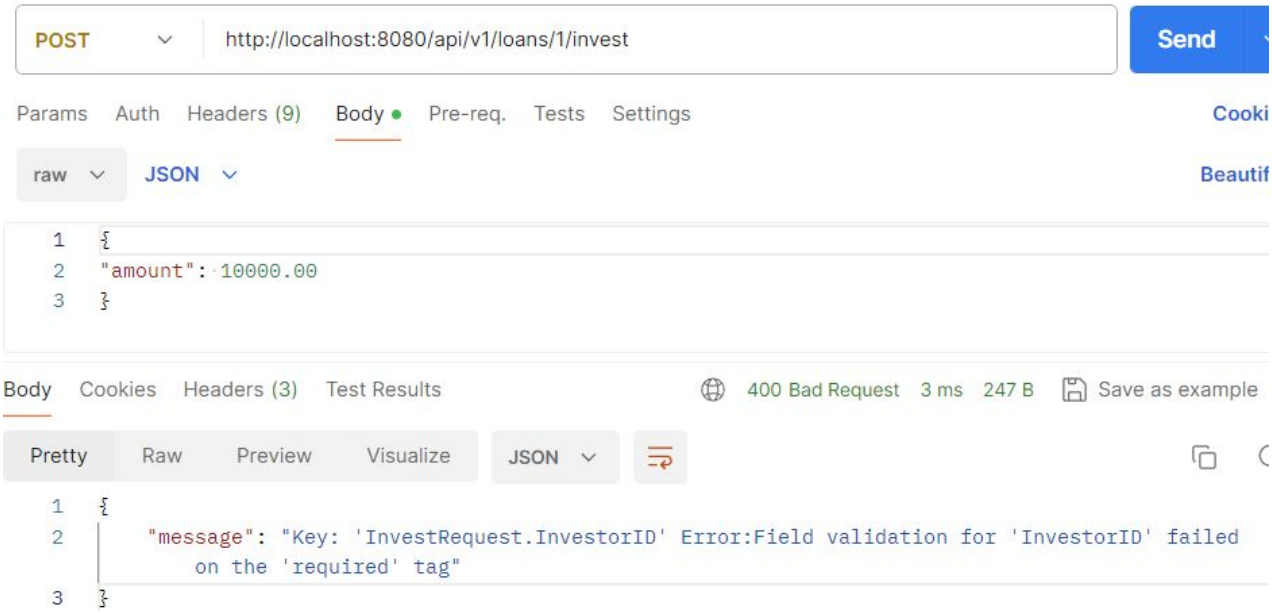
4.4 Invest Loan API

Test Case 1: Successful Investment（without status change）

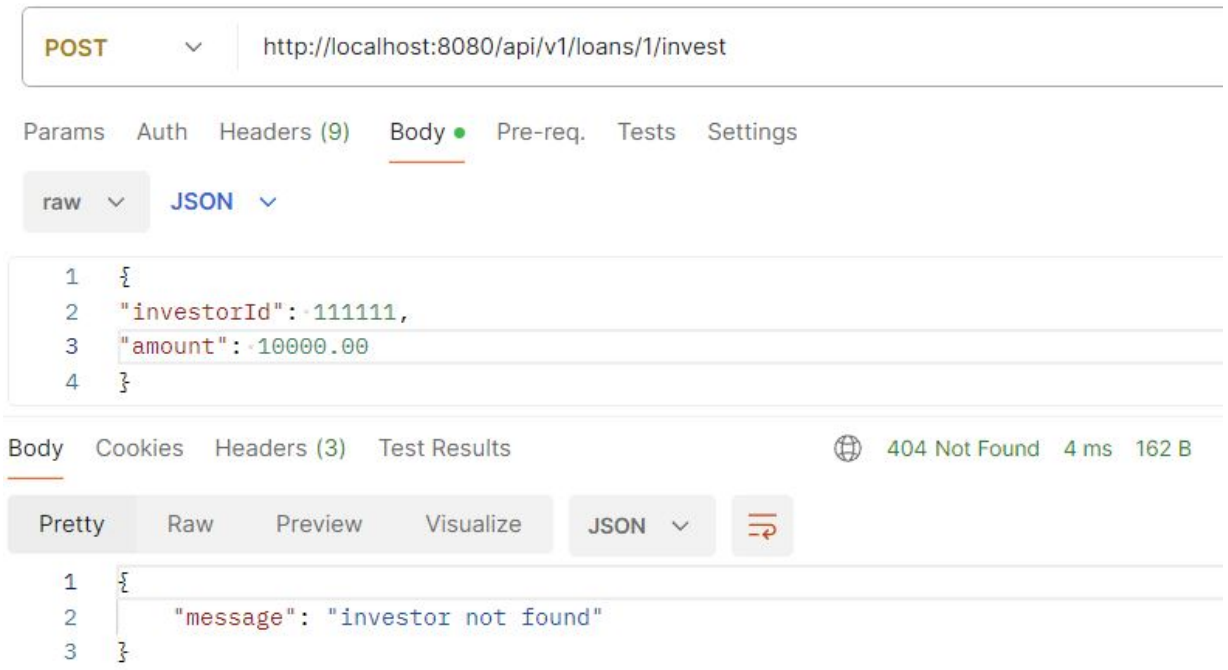
The sum of investment does not exceed the principal



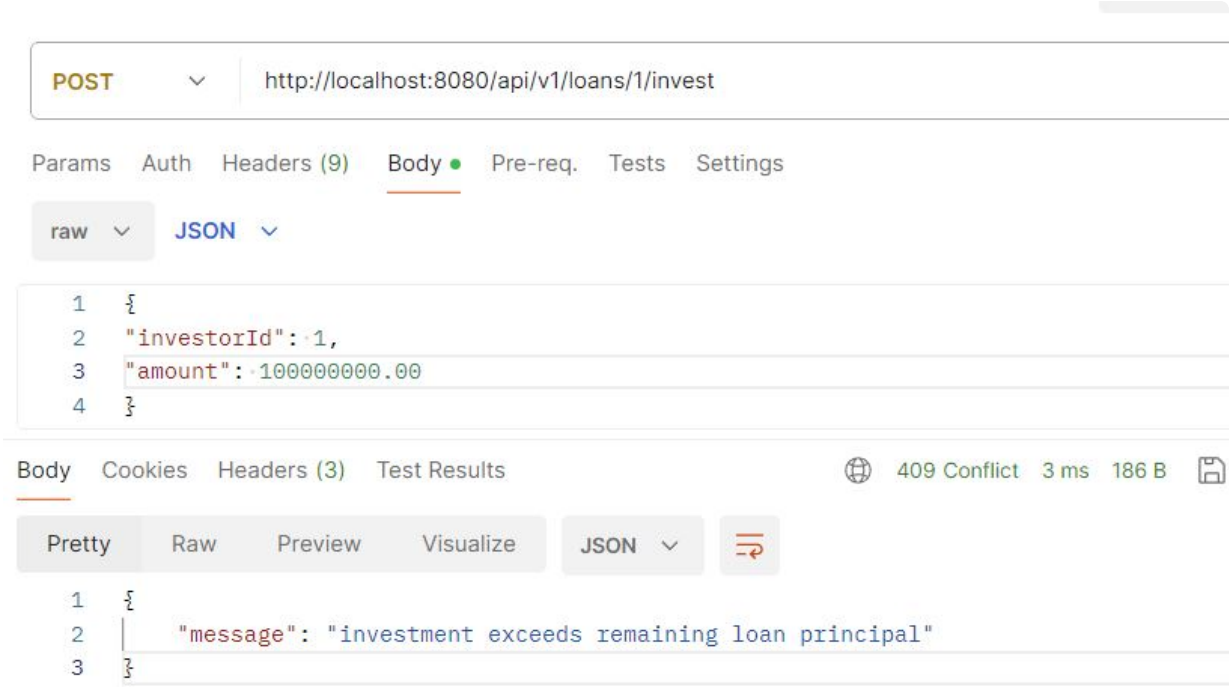
Test Case 2: Missing Required Parameters



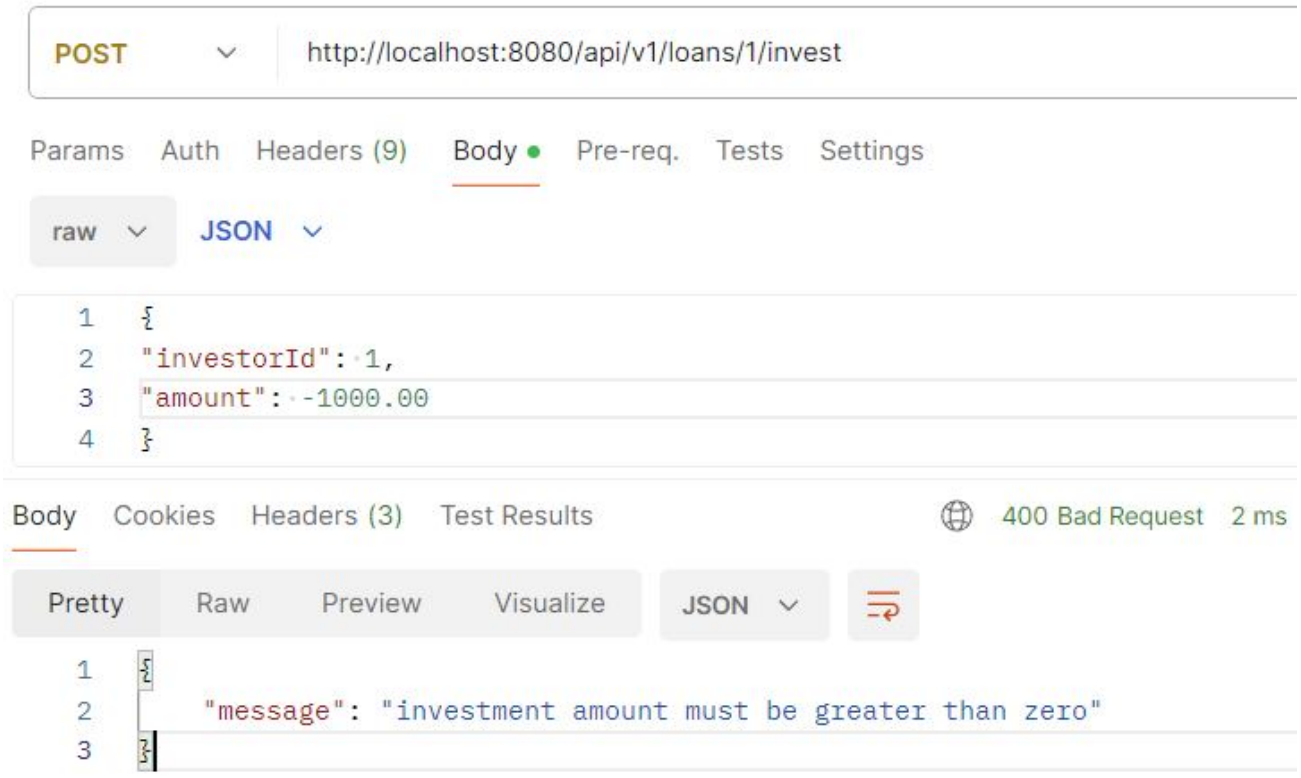
Test Case 3: Investor Does Not Exist



Test Case 4: Investment exceeds remaining loan principal

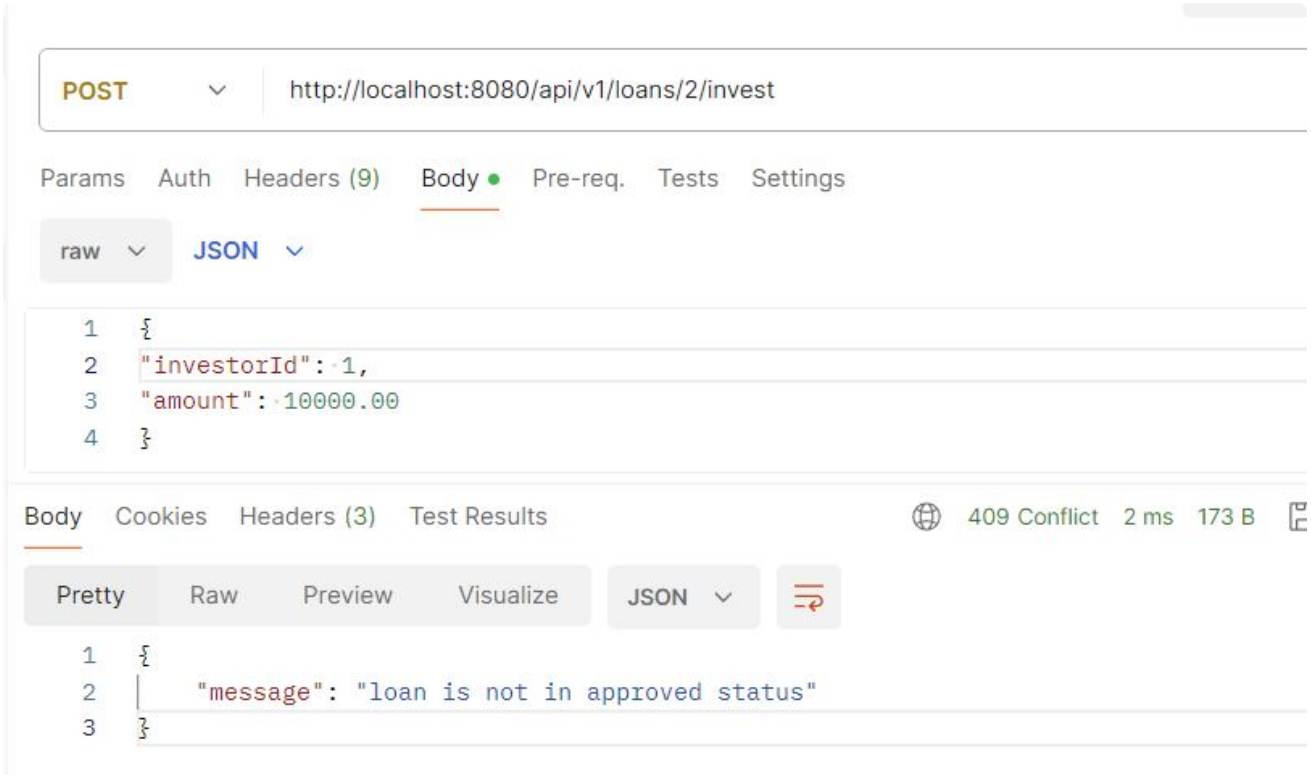


Test Case 5: Invalid Parameters



Test Case 6: Invalid Loan Status

This case(loan_id = 2) is in `PROPOSED` status, can not be invested.

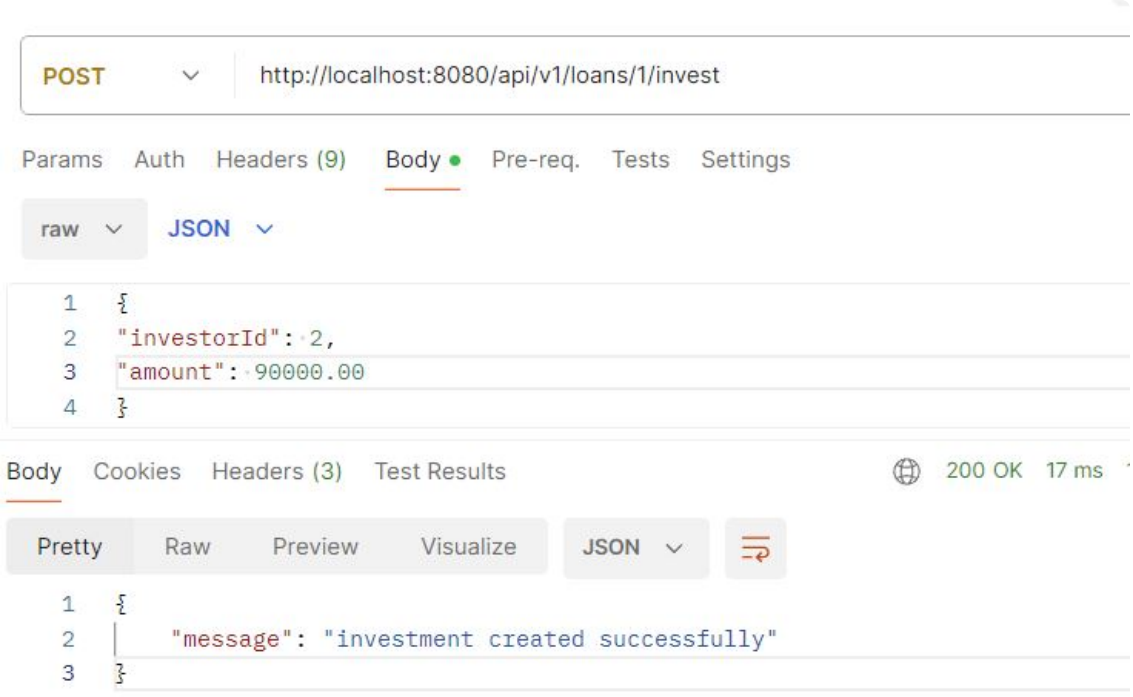


Test Case 7: Successful Investment（change status & send emails）

In this case (loan_id = 1), the total Investment Amount equals to Principal

The Loan status changed

The emails are sent (Mock send, with database record)




```
mysql> SELECT id,status
    -> FROM loans
    -> WHERE id = 1;
+----+----------+
| id | status   |
+----+----------+
|  1 | INVESTED |
+----+----------+
1 row in set (0.004 sec)
```

```
mysql> SELECT id, loan_id, investment_id, channel, recipient, agreement_letter_url, status
    -> FROM notification_logs
    -> WHERE loan_id = 1;
+----+---------+---------------+---------+---------------------+-----------------------------------------+--------+
| id | loan_id | investment_id | channel | recipient           | agreement_letter_url                    | status |
+----+---------+---------------+---------+---------------------+-----------------------------------------+--------+
|  1 |       1 |             1 | email   | investor1@example.com | https://example.com/photos/agreement.jpg | sent   |
|  2 |       1 |             2 | email   | investor2@example.com | https://example.com/photos/agreement.jpg | sent   |
+----+---------+---------------+---------+---------------------+-----------------------------------------+--------+
2 rows in set (0.003 sec)
```

Test Case 8: Email – Successful Scheduled Task

Set test case manually.

log_id = 1, status = pending, retry_count = 1.

```
mysql> SELECT id, loan_id, status, retry_count FROM notification_logs;
+----+---------+---------+-------------+
| id | loan_id | status  | retry_count |
+----+---------+---------+-------------+
|  1 |       1 | pending |           1 |
|  2 |       1 | sent    |           0 |
|  3 |       1 | sent    |           0 |
+----+---------+---------+-------------+
```

Wait 10 minutes, and the scheduled task triggered, and the email was sent.

```
mysql> SELECT id, loan_id, status, retry_count FROM notification_logs;
+----+---------+--------+-------------+
| id | loan_id | status | retry_count |
+----+---------+--------+-------------+
|  1 |       1 | sent   |           1 |
|  2 |       1 | sent   |           0 |
|  3 |       1 | sent   |           0 |
+----+---------+--------+-------------+
```

Test Case 8: Email – Invalid Scheduled Task

Set test case manually.

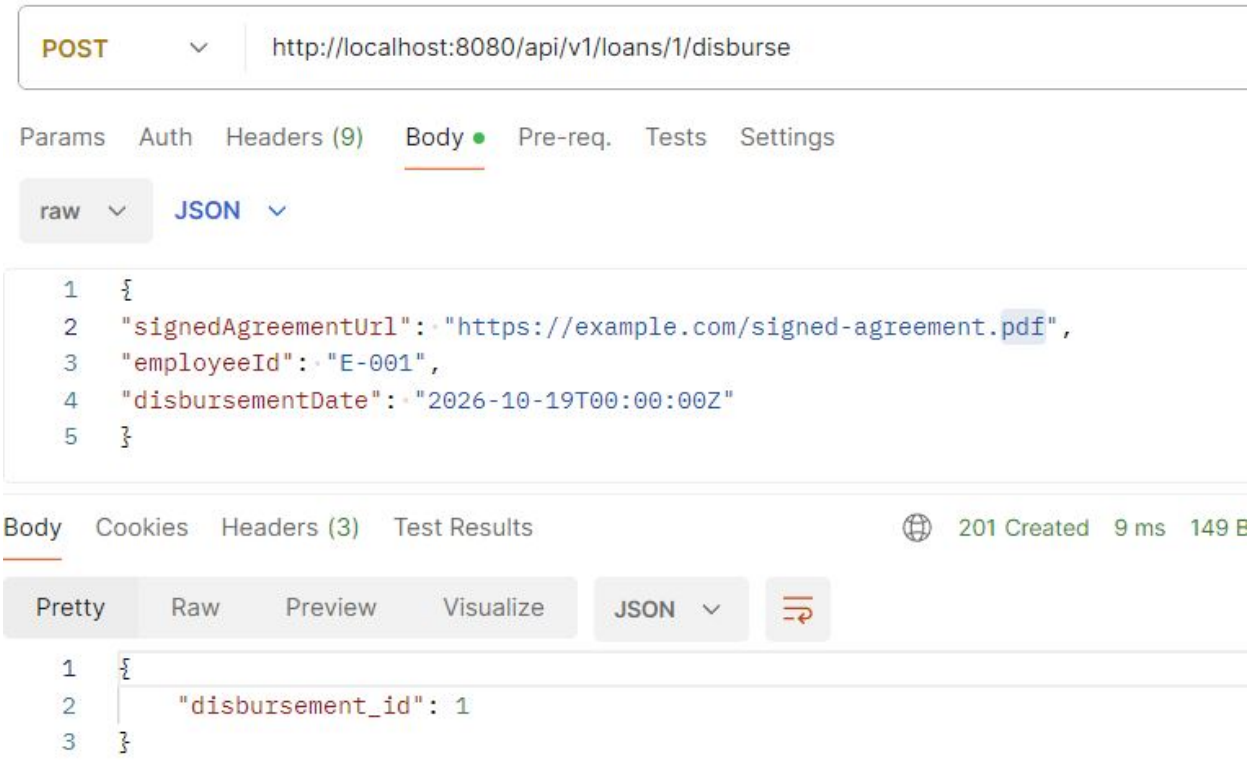
log_id = 2, status = pending, retry_count = 4.

```
mysql> SELECT id, loan_id, status, retry_count FROM notification_logs;
+----+---------+---------+-------------+
| id | loan_id | status  | retry_count |
+----+---------+---------+-------------+
|  1 |       1 | sent    |           1 |
|  2 |       1 | pending |           4 |
|  3 |       1 | sent    |           0 |
+----+---------+---------+-------------+
```

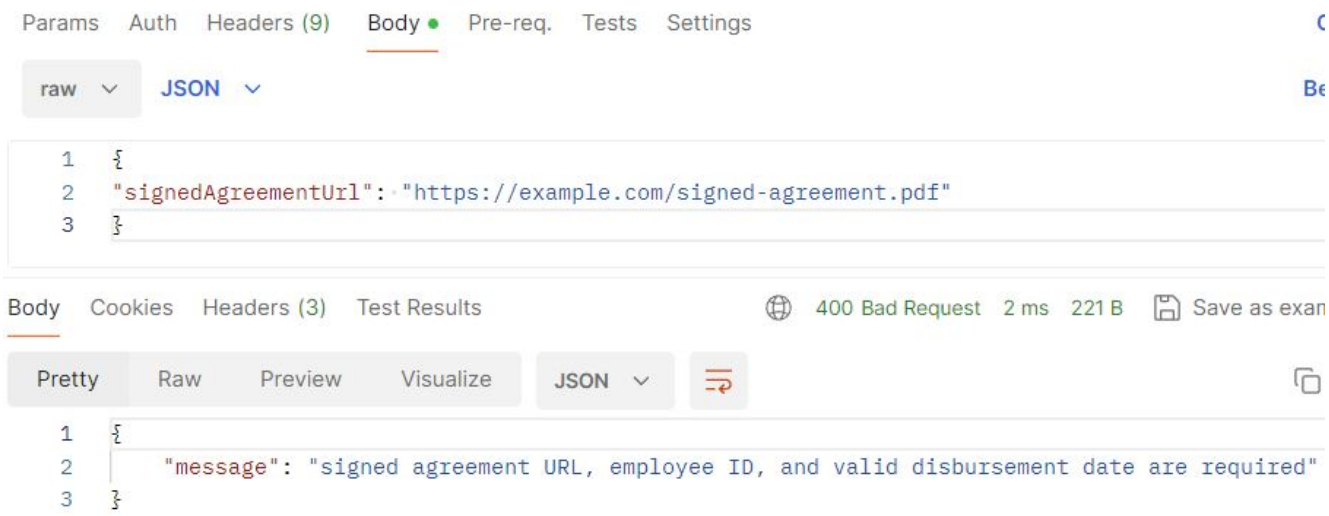
Retry count > 3, so the email will not be process anymore.

4.5 Disburse Loan API

Test Case 1: Successful Loan Disbursement

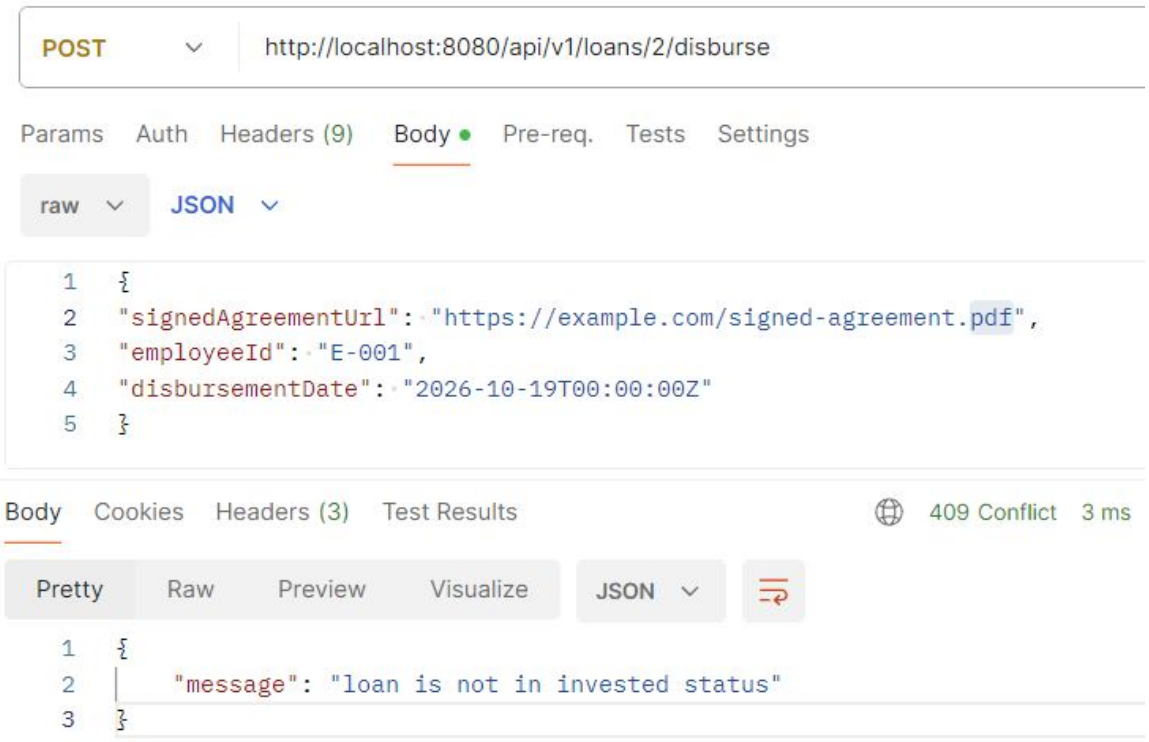


Test Case 2: Missing Required Parameters

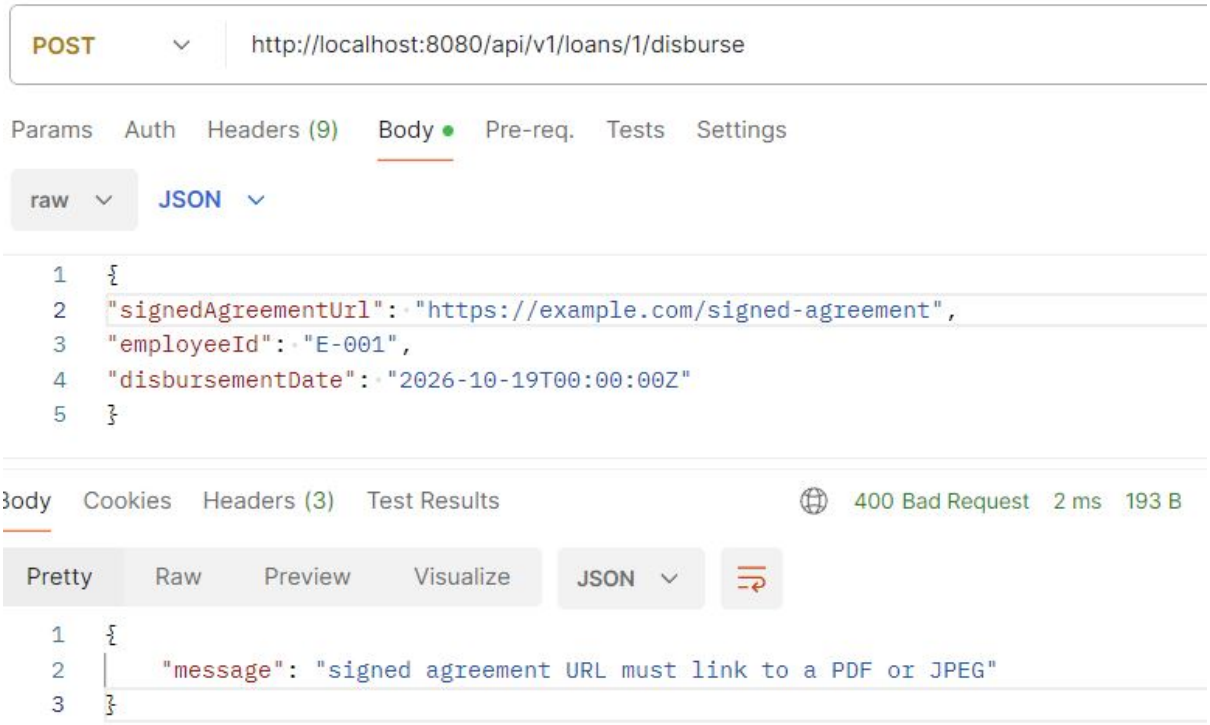


Test Case 3: Invalid Status

This case(loan_id = 2) is in `PROPOSED` status, can not be disbursed.



Test Case 4: Invalid URL



Additional Notes

Concurrency Control & Race Conditions

The investment workflow has been designed to handle concurrent requests. Database transactions and row–level locking are used to protect critical business operations and prevent data consistency issues, such as over–investment caused by concurrent requests.

Due to limitations of the local testing environment, dedicated concurrency and load testing have not been performed for this submission. In practice, these tests are typically conducted in a staging or dedicated test environment using specialized concurrency and performance testing tools.